# Linnest Typst API

Linnest exposes the `linnest.wasm` graph layout plugin through a small Typst API. The public surface is intentionally narrow:

- `graph` for construction, parsing, inspection, joins, and graph algorithms.
- `subgraph` for subgraph object construction and inspection.
- `layout` for the separate layout pass.
- `config` for default layout settings.

## Minimal Builder Example

```
#import "../src/lib.typ": graph, layout, subgraph

#let b = graph.builder(name: "demo")
#let (node: a, builder: b) = graph.node(b, name: "a")
#let (node: c, builder: b) = graph.node(b, name: "c")
#let b = graph.edge(b, source: (node: a), sink: (node: c))
#let g = graph.finish(b)
#let g = layout(g, seed: "2", steps: "5")
#let north = subgraph.compass(g, "n")
#let edges = graph.edges(g, subgraph: north)
#let dot = graph.dot(g)
```

## Graph Objects

Graph, builder, and subgraph objects are opaque zero-copy values. Build or parse graph objects with `graph`, transform graph objects with `layout`, and pass objects back to `graph` or `subgraph` for inspection.

- `graph.parse(input)` parses one or more DOT digraphs and returns an array of graph objects.
- `graph.build(..)` constructs one graph object from named arguments. This is convenience sugar over the builder functions.
- `graph.builder(..)` starts a builder object.
- `graph.node(builder, ..)` returns `(node: index, builder: builder)`.
- `graph.edge(builder, ..)` returns the updated builder.
- `graph.finish(builder)` turns a builder into a graph.
- `layout(graph, seed: "2", steps: "5", ..)` runs layout as an explicit second step. Its settings are named parameters so calls stay descriptive and Tidy can document each field.
- `graph.dot(graph)` returns a DOT string for inspection or export.

## Builder Spec

The builder API is the preferred construction API when nodes need to be reused. Use destructuring to keep the builder value moving:

```
#let (node: a, builder: b) = graph.node(b, name: "a")
```

`graph.edge` accepts `source` and `sink` endpoint dictionaries. An endpoint can contain `node`, `statement`, `id`, `port_label`, `compass`, and `in_subgraph`. Set `source: none` or `sink: none` to create an external half edge.

`graph.build` accepts the same data in one dictionary:

```
#let g = graph.build(
  name: "demo",
  statements: (full_num: "x + y"),
  node-statements: (shape: "circle"),
  edge-statements: (color: "black"),
  nodes: ((name: "a"), (name: "b")),
  edges: ((
    source: (node: 0, compass: "e"),
    sink: (node: 1, compass: "w"),
    statements: (label: "ab"),
  ),),
)
```

## Graph Queries

`graph.info(g)` returns graph metadata. `graph.nodes(g)` returns node records, and `graph.edges(g)` returns edge records. Pass `subgraph: sg` to filter nodes or edges by an subgraph object.

`graph.join(left, right, key: "statement")` joins matching dangling half edges. The key is read from half-edge data and can be `"statement"`, `"port_label"`, `"compass"`, or `"id"`.

`graph.cycles(g)` returns subgraph objects for a cycle basis. `graph.forests(g)` returns subgraph objects for spanning forests.

## Layout Model

`layout` starts from a traversal-tree placement, then optimizes the positions of graph nodes and edge control points. The initial tree spacing is

$$L = \lambda \sqrt{\frac{WH}{\max(n, 1)}}$$

,

with horizontal spacing $\tau_x L$ and vertical spacing $\tau_y L$. Here $\lambda$ is `length_scale`, $W$ is `viewport_w`, $H$ is `viewport_h`, $\tau_x$ is `tree_dx`, and $\tau_y$ is `tree_dy`. These fields set the geometry scale for both layout modes.

The shared spring/charge model uses the following coefficients:

$$c_{\mathrm{vv}} = \beta L^2$$

$$c_{\mathrm{ev}} = \beta \gamma_{\mathrm{ev}} L^2$$

$$c_{\mathrm{ee}} = \beta \gamma_{\mathrm{ee}} L^2$$

$$c_{\mathrm{center}} = \beta g_{\mathrm{center}} L^2$$

$$c_{\mathrm{dangling}} = \beta \gamma_{\mathrm{dangling}} L^2$$

The Typst parameter names are `beta` for $\beta$, `gamma_ev` for $\gamma_{\mathrm{ev}}$, `gamma_ee` for $\gamma_{\mathrm{ee}}$, `g_center` for $g_{\mathrm{center}}$, and `gamma_dangling` for $\gamma_{\mathrm{dangling}}$. The spring stiffness $k$ is `k_spring`, and the softening constant $\varepsilon$ is `eps`.

In `layout_algo: "anneal"`, linnest minimizes an energy:

$$\frac{c_{\mathrm{vv}}}{d(v_i, v_j) + \varepsilon} + \sum_{i,e} \frac{c_{\mathrm{ev}}}{d(v_i, e) + \varepsilon} + \sum_{(v,e) \text{ incident}} \frac{1}{2} k(\ell_e - d(v, e))^2 + \sum_{\text{local edge pairs}} \frac{1}{2} \frac{c_{\mathrm{ee}}}{d(e_i, e_j) + \varepsilon} + \sum_{\text{dangling pairs}} \frac{1}{2} \frac{c_{\mathrm{dangling}}}{d(e_i, e_j) + \varepsilon} + \sum_i \frac{1}{2} \frac{c_{\mathrm{center}}}{d(v_i, 0) + \varepsilon} +$$

.

Here $p_{\mathrm{cross}}$ is `crossing_penalty` and $N_{\mathrm{cross}}$ is the number of detected edge crossings. `temp`, `step`, `seed`, `steps`, `epochs`, `cool`, `accept_floor`, `step_shrink`, and `incremental_energy` belong to this simulated annealing mode. `crossing_penalty` is also anneal-only; force mode does not currently add a crossing force.

In `layout_algo: "force"`, linnest applies the direct forces corresponding to the same vertex-vertex, edge-vertex, incidence spring, local edge-edge, dangling-edge, and center terms. `step` is the integration step, `delta` clamps per-step movement, `steps` and `epochs` set the iteration budget, `cool` shrinks the step after each epoch, and `early_tol` stops when movement is small. `z_spring` and `z_spring_growth` are force-only helpers: the integrator gives points temporary z coordinates to break overlaps and pulls them back toward the 2D plane.

`directional_force` is applied in both modes as an extra bias derived from pin/port direction constraints.

After either graph layout mode, labels are relaxed separately. If $L_l$ is the label target distance and $q_l$ is the label repulsion strength, then $L_l = \alpha_l L$ and $q_l = \beta_l L^2$. The Typst names are `label_length_scale` for $\alpha_l$ and `label_charge` for $\beta_l$. `label_spring` is the spring constant pulling each label toward its target. `label_steps`, `label_step`, `label_early_tol`, and `label_max_delta_scale` control the label relaxation iteration.

## Subgraphs

Subgraph objects are opaque zero-copy values.

- `subgraph.label(g, label)` constructs a subgraph from a base62 label.
- `subgraph.bits(g, bits)` constructs a subgraph from a boolean hedge array.
- `subgraph.compass(g, compass)` selects half edges with a DOT compass point.
- `subgraph.to-label(sg)` returns the base62 label.
- `subgraph.hedges(sg)` returns included hedge indices.
- `subgraph.contains(sg, hedge)` tests hedge membership.

## Generated Reference

- layout()

### Variables

- config
- graph
- subgraph

### layout

Apply the linnest layout pass to a graph object.

This is intentionally a second step: construct or parse a graph first, then call `layout`. Set `layout_algo` to `"force"` for deterministic force integration or `"anneal"` for simulated annealing.

```
#let g = graph.build(
  nodes: ((name: "a"), (name: "b")),
  edges: ((source: (node: 0), sink: (node:
1)),),
)
#let g = layout(g, seed: "2", steps: "5")
#graph.edges(g).map(edge => edge.pos)
```

```
((x: 0.018342069370456824, y:
16.866953853092777),)
```

**Parameters**

```
layout(
  graph: bytes,
  viewport_w: float,
  viewport_h: float,
  tree_dx: float,
  tree_dy: float,
  steps: int,
  seed: int,
  step: float,
  step_shrink: float,
  cool: float,
  accept_floor: float,
  early_tol: float,
  temp: float,
  delta: float,
  beta: float,
  k_spring: float,
  g_center: float,
  epochs: int,
  crossing_penalty: float,
  gamma_dangling: float,
  gamma_ee: float,
  directional_force: float,
  label_length_scale: float,
  label_spring: float,
  label_charge: float,
  label_steps: int,
  label_step: float,
  label_early_tol: float,
  label_max_delta_scale: float,
  gamma_ev: float,
  eps: float,
  incremental_energy: bool,
  layout_algo: string,
  z_spring: float,
  z_spring_growth: float,
  length_scale: float
) -> bytes
```

**graph** `bytes`

Graph object returned by `graph.build`, `graph.finish`, or `graph.parse`.

**viewport_w** `float`

Width of the layout viewport used to derive the natural spring length. Applies to both `"force"` and `"anneal"`.

Default: `config.viewport_w`

**viewport_h** `float`

Height of the layout viewport used to derive the natural spring length. Applies to both `"force"` and `"anneal"`.

Default: `config.viewport_h`

**tree_dx** `float`

Horizontal spacing multiplier for the initial traversal-tree placement. Applies before both `"force"` and `"anneal"`.

Default: `config.tree_dx`

**tree_dy** `float`

Vertical spacing multiplier for the initial traversal-tree placement. Applies before both `"force"` and `"anneal"`.

Default: `config.tree_dy`

**steps** `int`

Iterations per epoch. In `"force"` mode this is the number of force integration steps; in `"anneal"` mode this is the number of proposals per temperature epoch.

Default: `config.steps`

**seed** `int`

Seed for deterministic initialization, force-mode jitter, and annealing proposals. Applies to both modes.

Default: `config.seed`

**step** `float`

Initial movement scale. `"force"` multiplies computed forces by this value; `"anneal"` uses it as the proposal step size.

Default: `config.step`

**step_shrink** `float`

Anneal-only step shrink factor, applied when an epoch's acceptance ratio falls below `accept_floor`.

Default: `config.step_shrink`

**cool** `float`

Cooling factor applied once per epoch. `"anneal"` cools `temp`; `"force"` shrinks `step`.

Default: `config.cool`

**accept_floor** `float`

Anneal-only acceptance-ratio threshold below which `step` is shrunk by `step_shrink`.

Default: `config.accept_floor`

**early_tol** `float`

Force-only early stop threshold for maximum movement in one step. The annealing schedule stores this value but does not currently use it for stopping.

Default: `config.early_tol`

**temp** `float`

Anneal-only initial temperature used in the Metropolis acceptance test.

Default: `config.temp`

**delta** `float`

Force-mode maximum movement clamp per point and per step.

Default: `config.delta`

**beta** `float`

Base repulsion strength for vertex-vertex interactions. Also scales `gamma_ev`, `gamma_ee`, `gamma_dangling`, and `g_center`. Applies to both modes through the shared spring energy.

Default: `config.beta`

**k_spring** `float`

Spring stiffness for node-to-edge incidence lengths. Applies to both modes.

Default: `config.k_spring`

**g_center** `float`

Centering strength relative to `beta`. Applies to both modes.

Default: `config.g_center`

**epochs** `int`

Number of epochs. Both modes run up to `steps` iterations inside each epoch.

Default: `config.epochs`

**crossing_penalty** `float`

Anneal-only fixed energy penalty per detected edge crossing. The direct force integrator does not currently add a crossing force.

Default: `config.crossing_penalty`

**gamma_dangling** `float`

Repulsion for dangling half edges, relative to `beta`. Applies to both modes through the shared spring energy.

Default: `config.gamma_dangling`

**gamma_ee** `float`

Local edge-edge repulsion, relative to `beta`. Applies to both modes.

Default: `config.gamma_ee`

**directional_force** `float`

Bias that pushes points in directions implied by pin/port constraints. Applies to both modes.

Default: `config.directional_force`

**label_length_scale** `float`

Edge-label target offset as a multiple of the graph spring length. Label layout runs after both graph layout modes.

Default: `config.label_length_scale`

**label_spring** `float`

Spring strength pulling each label toward its target offset. Label layout runs after both modes.

Default: `config.label_spring`

**label_charge** `float`

Repulsion strength between labels and graph points, scaled by spring length squared. Label layout runs after both modes.

Default: `config.label_charge`

**label_steps** `int`

Maximum number of post-layout label relaxation steps. Set to `"0"` to skip label placement. Applies after both modes.

Default: `config.label_steps`

**label_step** `float`

Label relaxation step size. Applies after both modes.

Default: `config.label_step`

**label_early_tol** `float`

Label relaxation early stop threshold. Applies after both modes.

Default: `config.label_early_tol`

**label_max_delta_scale** `float`

Label movement clamp as a multiple of spring length. Applies after both modes.

Default: `config.label_max_delta_scale`

**gamma_ev** `float`

Edge-vertex repulsion, relative to `beta`. Applies to both modes.

Default: `config.gamma_ev`

**eps** `float`

Softening epsilon used in inverse-square force/energy terms. Applies to both modes.

Default: `config.eps`

**incremental_energy** `bool`

Whether annealing updates cached energy by local deltas. This is anneal-only; force mode computes direct forces instead of energies.

Default: `config.incremental_energy`

**layout_algo** `string`

Layout algorithm. Use `"force"` for direct force integration or `"anneal"` for simulated annealing against the spring energy.

Default: `config.layout_algo`

**z_spring** `float`

Force-only spring pulling temporary z coordinates back toward the layout plane. Use with `z_spring_growth` to help separate overlapping points during integration.

Default: `config.z_spring`

**z_spring_growth** `float`

Force-only per-epoch multiplier for `z_spring`.

Default: `config.z_spring_growth`

**length_scale** `float`

Natural spring-length multiplier. This scales the graph's preferred edge length and the repulsive coefficients derived from it. Applies to both modes.

Default: `config.length_scale`

**config** `dictionary`

Default layout configuration used by `layout()`.

These are the defaults for `layout()`'s named settings. Numeric values are native Typst integers/floats here and are converted to strings only when the wrapper sends settings to the plugin.

**graph** `module`

Graph namespace.

Graph objects are opaque zero-copy values. Keep them opaque and pass them back to this module, `subgraph`, or `layout()`.

**subgraph** `module`

Subgraph namespace.

Subgraph objects are opaque zero-copy values. They can be passed to `graph.nodes` and `graph.edges`.

## graph

- build()
- builder()
- cycles()
- dot()
- edge()
- edges()
- finish()
- forests()
- info()
- join()
- node()
- nodes()
- parse()

### build

Build one graph object from node and edge arrays.

This is convenience sugar over `builder()`, `node()`, `edge()`, and `finish()`.

```
#let g = graph.build(
  name: "demo",
  nodes: ((name: "a"), (name: "b")),
  edges: ((source: (node: 0), sink: (node:
1)),),
)
#graph.info(g).name
```

demo

**Parameters**

```
build(
  name: none string,
  statements: dictionary,
  edge-statements: dictionary,
  node-statements: dictionary,
  nodes: array,
  edges: array
) -> bytes
```

**name** `none` or `string`

Graph name.

Default: none

**statements** `dictionary`

Graph-level DOT statements.

Default: `(:)`

**edge-statements** `dictionary`

Default DOT statements for edges.

Default: `(:)`

**node-statements** `dictionary`

Default DOT statements for nodes.

Default: `(:)`

**nodes** `array`

Node specifications. Each node may define `name`, `index`, and `statements`. This matches the per-node `statements` parameter accepted by `node()`.

Default: `()`

**edges** `array`

Edge specifications. Each edge may define `source`, `sink`, `orientation`, `flow`, `id`, and `statements`. The `source` and `sink` fields use the same endpoint dictionaries accepted by `edge()`.

Default: `()`

### builder

Create a builder object.

`statements`, `node-statements`, and `edge-statements` set graph-level DOT attributes that are carried into the finished graph.

```
#let b = graph.builder(name: "demo", statements:
(full_num: "x + y"))
#let (node: a, builder: b) = graph.node(b, name:
"a")
#a
```

```
0
```

### Parameters

```
builder(
  name,
  statements,
  node-statements,
  edge-statements
) -> bytes
```

### cycles

Return subgraph objects for the graph's cycle basis.

```
#let g = graph.build(
  nodes: ((name: "a"), (name: "b")),
  edges: ((source: (node: 0), sink: (node:
1)),),
)
#graph.cycles(g).len()
```

```
0
```

**Parameters**

```
cycles(graph) -> array
```

### dot

Serialize a graph object to DOT.

```
#let g = graph.build(
  name: "demo",
  nodes: ((name: "a"), (name: "b")),
  edges: ((source: (node: 0), sink: (node:
1)),),
)
#graph.dot(g).contains("digraph demo")
```

```
true
```

**Parameters**

```
dot(graph) -> string
```

### edge

Add a paired or external edge to a builder object.

Set `source` to `none` or `sink` to `none` for an external edge. Endpoint dictionaries accept `node`, `statement`, `id`, `port_label`, `compass`, and `in_subgraph`.

```
#let b = graph.builder(name: "demo")
#let (node: a, builder: b) = graph.node(b, name:
"a")
#let (node: c, builder: b) = graph.node(b, name:
"c")
#let b = graph.edge(
  b,
  source: (node: a, compass: "e"),
  sink: (node: c, compass: "w"),
  statements: (label: "a-c"),
)
#graph.edges(graph.finish(b)).len()
```

```
1
```

**Parameters**

```
edge(
  builder,
  source,
  sink,
  orientation,
  flow,
  id,
  statements
) -> bytes
```

### edges

Return edge records, optionally filtered by an subgraph object.

```
#let g = graph.build(
  nodes: ((name: "a"), (name: "b")),
  edges: ((source: (node: 0, compass: "e"),
sink: (node: 1)),),
)
#let east = subgraph.compass(g, "e")
#graph.edges(g, subgraph: east).len()
```

```
1
```

**Parameters**

```
edges(
  graph,
  subgraph
) -> array
```

### finish

Finish a builder object into a graph object.

```
#let b = graph.builder(name: "demo")
#let (node: a, builder: b) = graph.node(b, name:
"a")
#let g = graph.finish(b)
#graph.info(g).name
```

```
demo
```

**Parameters**

```
finish(builder) -> bytes
```

### forests

Return subgraph objects for the graph's spanning forests.

```
#let g = graph.build(
  nodes: ((name: "a"), (name: "b")),
  edges: ((source: (node: 0), sink: (node:
1)),),
)
#graph.forests(g).len()
```

```
1
```

**Parameters**

```
forests(graph) -> array
```

### info

Return graph metadata.

The result has `name`, `global_statements`, `edge_statements`, and `node_statements`.

```
#let g = graph.build(name: "demo", nodes:
((name: "a"),))
#graph.info(g).name
```

```
demo
```

**Parameters**

```
info(graph) -> dictionary
```

### join

Join two graphs by matching dangling half-edge data on `key`.

Supported key values are `"statement"`, `"port_label"`, `"compass"`, and `"id"`.

```
#let left = graph.build(
  nodes: ((name: "a"),),
  edges: ((source: none, sink: (node: 0,
statement: "j")),),
)
#let right = graph.build(
  nodes: ((name: "b"),),
  edges: ((source: (node: 0, statement: "j"),
sink: none),),
)
#graph.edges(graph.join(left, right, key:
"statement")).len()
```

```
1
```

#### Parameters

```
join(
  left,
  right,
  key
) -> bytes
```

### node

Add a node to a builder object.

Returns a dictionary with fields `node` and `builder`, so callers can write `#let (node: a, builder: b) = graph.node(b, name: "a")`. The returned builder is the value to pass to the next `node()`, `edge()`, or `finish()` call.

```
#let b = graph.builder(name: "demo")
#let (node: a, builder: b) = graph.node(b, name:
"a")
#let (node: c, builder: b) = graph.node(b, name:
"c")
#repr((a, c))
```

```
(0, 1)
```

#### Parameters

```
node(
  builder,
  name,
  index,
  statements
) -> dictionary
```

### nodes

Return node records, optionally filtered by an subgraph object.

```
#let g = graph.build(nodes: ((name: "a"), (name:
"b")))
#graph.nodes(g).map(node => node.name).join(",
")
```

```
a, b
```

**Parameters**

```
nodes(
  graph,
  subgraph
) -> array
```

### parse

Parse one or more DOT digraphs into graph objects.

```
#let graphs = graph.parse("digraph first { a -> b }")
#graphs.len()
```

```
1
```

**Parameters**

```
parse(input) -> array
```

## subgraph

- bits()
- compass()
- contains()
- hedges()
- label()
- to-label()

### bits

Construct an subgraph object from a boolean hedge array.

```
#let g = graph.build(
  nodes: ((name: "a"), (name: "b")),
  edges: ((source: (node: 0), sink: (node: 1)),),
)
#let first = subgraph.bits(g, (true, false))
#subgraph.contains(first, 0)
```

```
true
```

**Parameters**

```
bits(
  graph,
  bits
) -> bytes
```

### compass

Construct an subgraph object from a DOT compass point such as `"n"` or `"s"`.

```
#let g = graph.build(
  nodes: ((name: "a"), (name: "b")),
  edges: ((source: (node: 0, compass: "e"), sink: (node: 1)),),
)
#subgraph.hedges(subgraph.compass(g, "e")).len()
```

```
1
```

**Parameters**

```
compass(
  graph,
  compass
) -> bytes
```

### contains

Test whether an subgraph object includes a hedge.

```
#let g = graph.build(
  nodes: ((name: "a"), (name: "b")),
  edges: ((source: (node: 0), sink: (node:
1)),),
)
#subgraph.contains(subgraph.bits(g, (true,
false)), 0)
```

```
true
```

**Parameters**

```
contains(
  subgraph,
  hedge
) -> bool
```

### hedges

Return the hedge indices included in an subgraph object.

```
#let g = graph.build(
  nodes: ((name: "a"), (name: "b")),
  edges: ((source: (node: 0), sink: (node:
1)),),
)
#subgraph.hedges(subgraph.bits(g, (true,
false)))
```

```
(0,)
```

**Parameters**

```
hedges(subgraph) -> array
```

### label

Construct an subgraph object from a base62 label.

```
#let g = graph.build(
  nodes: ((name: "a"), (name: "b")),
  edges: ((source: (node: 0, compass: "e"),
sink: (node: 1)),),
)
#let east = subgraph.compass(g, "e")
#let same = subgraph.label(g, subgraph.to-
label(east))
#subgraph.hedges(same).len()
```

```
1
```

**Parameters**

```
label(
  graph,
  label
) -> bytes
```

### to-label

Convert an subgraph object to its base62 label.

```
#let g = graph.build(
  nodes: ((name: "a"), (name: "b")),
  edges: ((source: (node: 0), sink: (node:
1)),),
)
#subgraph.to-label(subgraph.bits(g, (true,
false)))
```

```
1
```

**Parameters**

```
to-label(subgraph) -> string
```